



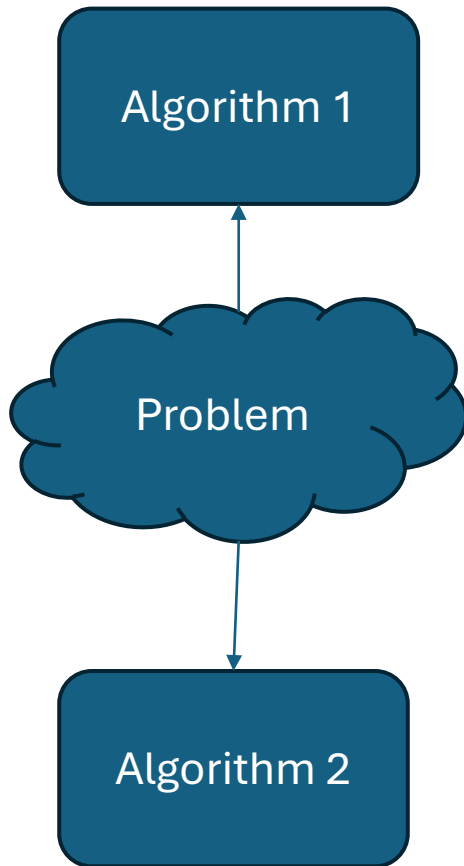
Analysis of iterative algorithms

Dr. Obed Noé Sámano Abonce



Analysis of algorithms

Comparing algorithms



We have two algorithms that solve the exact same problem ...

How can we compare them?

Efficiency:

- Execution time
- Memory usage

Run and measure?

Analysis of algorithms

Measuring execution time

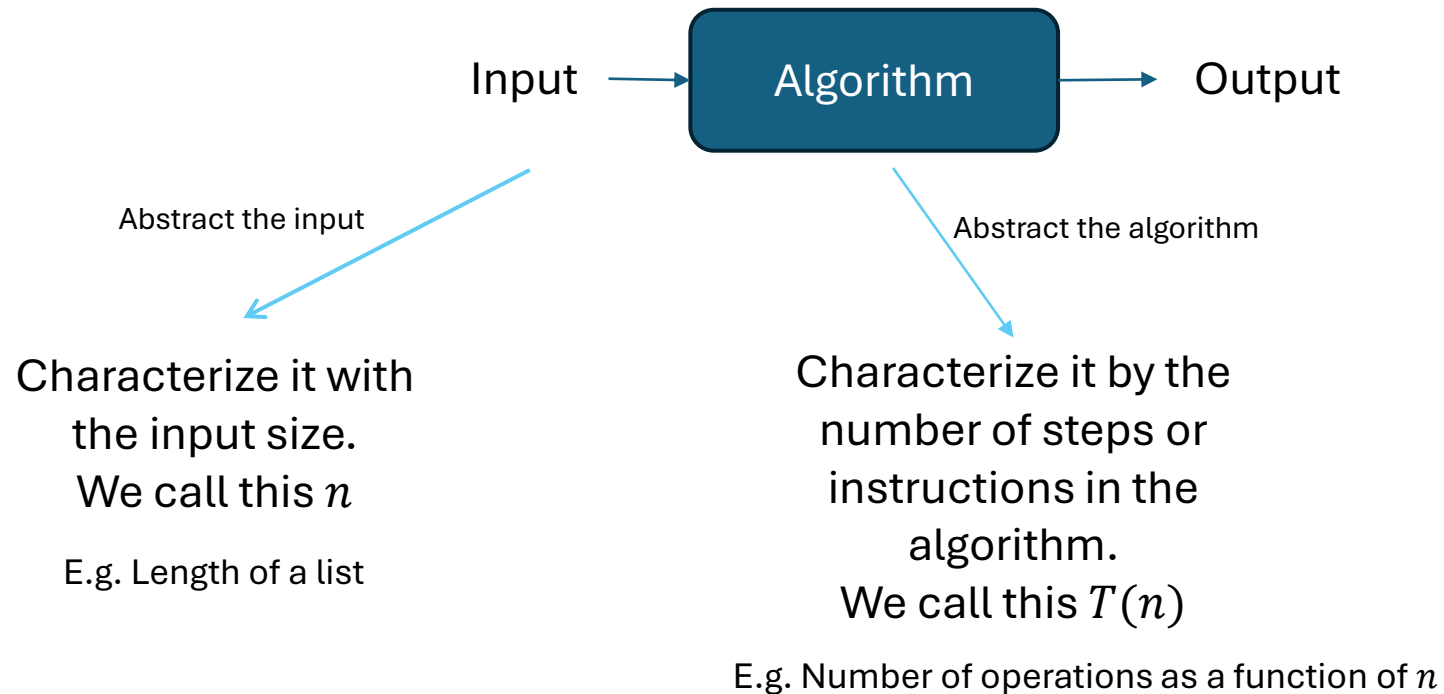
- Time execution depends on:
 1. The **input** of the algorithm: size and characteristics.
 2. Logic and structure of the algorithm.
 3. Code quality generated by the compiler.
 4. Speed of the host machine (hardware).

We need a way to compare algorithms that is independent on hardware!

Analysis of algorithms

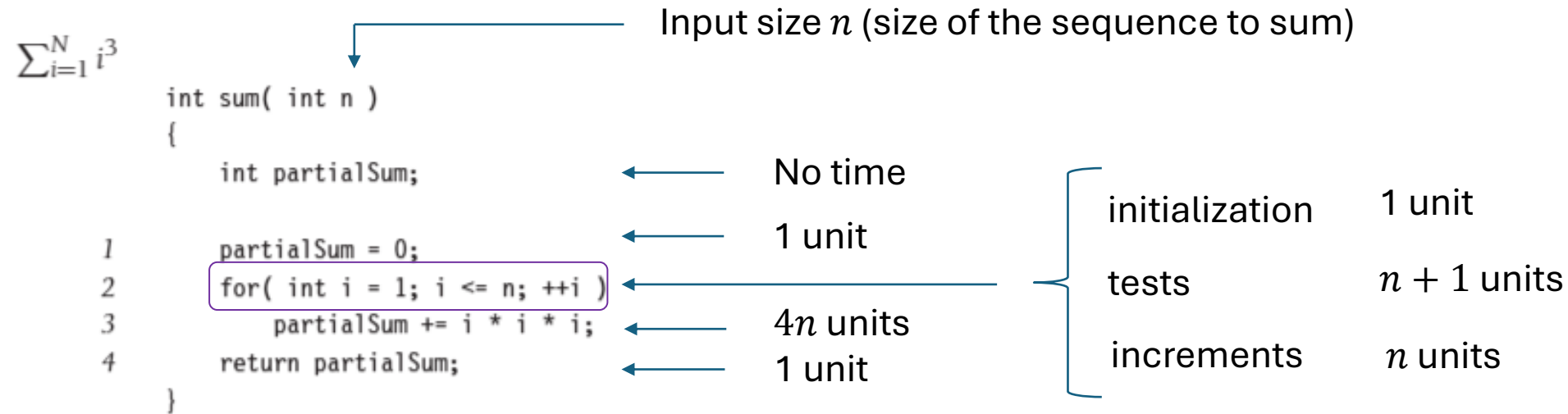
Comparing algorithms

Abstracting input and algorithm ...



Analysis of algorithms

Example



Input size → n

Algorithm → the number of operations:

$$T(N) = 6n + 4$$

- Not all the examples are that easy.
- Often, it's hard to find the exact $T(n)$
- We often simplify to the grow rate (asymptotic analysis).

Analysis of algorithms

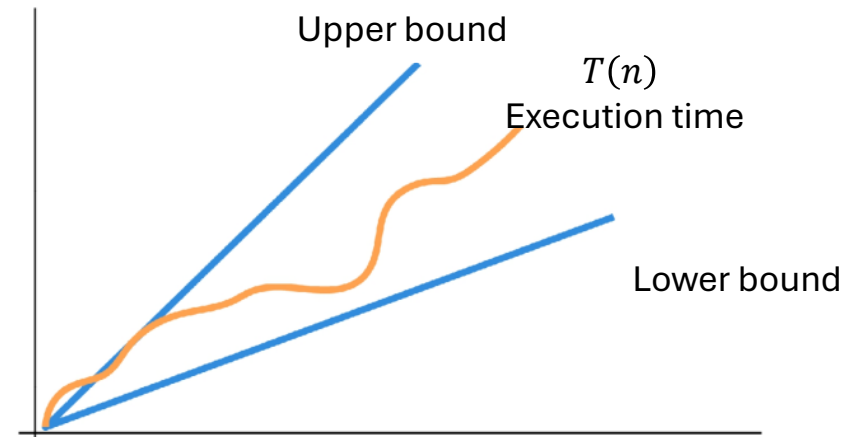
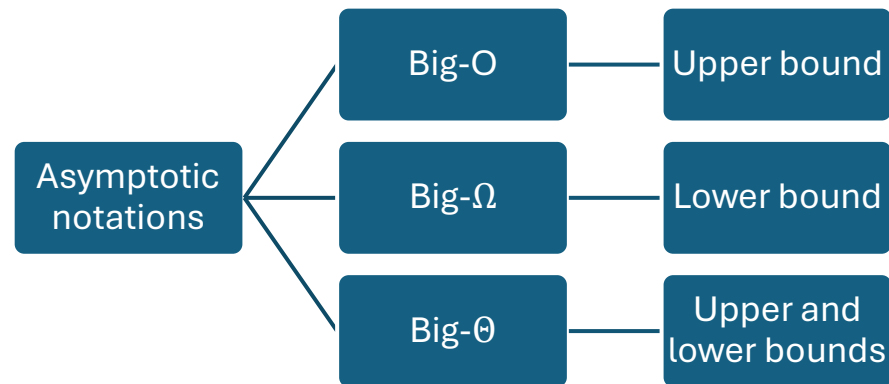
Asymptotic notation

- We are usually interested in the behaviour when n is big.
- Therefore, we use asymptotic analysis.
- **Temporal complexity.** Time required by an algorithm, expressed in terms of a function that depends on the input size, i.e. $T(n)$.
- **Asymptotic temporal complexity:** Grow behaviour of $T(n)$ as $n \rightarrow \infty$, ignoring constants and low-order terms.

Analysis of algorithms

Asymptotic notation

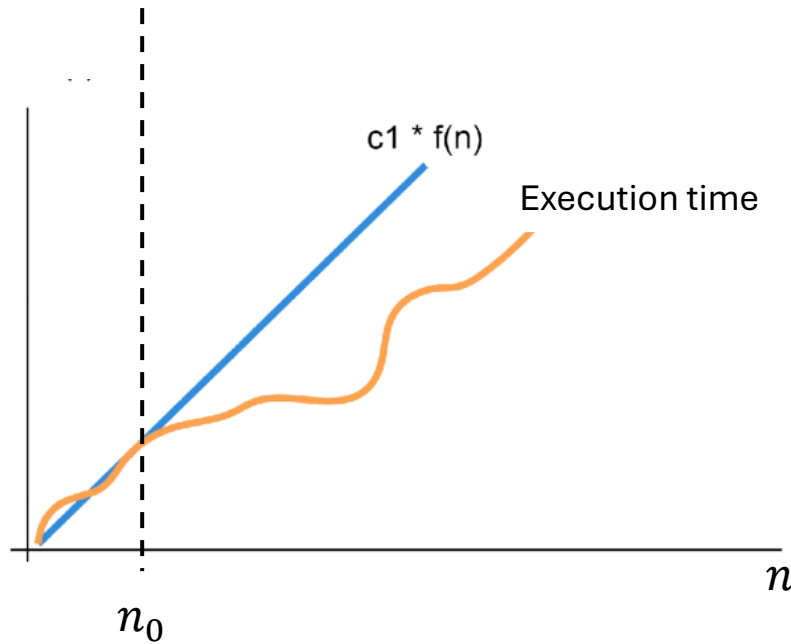
- Depending on the bounds considered, we can distinguish three types of notations:



Big-O notation

Definition

The function $T(n)$ is said to be “of order $g(n)$ ” [$O(g(n))$] if there exist positive constants c and n_0 such that $T(n) \leq cg(n)$ whenever $n \geq n_0$.



Examples:

- $n+5$ is **$O(n)$** because $n+5 \leq 2n$ for all $n \geq 5$
- $(n-1)^2$ is **$O(n^2)$** because $(n-1)^2 \leq 4n^2$ for all $n \geq 1$
- $(n-1)^2$ **NO** es **$O(n)$** since for any constant $c > 1$, $(n-1)^2 \leq c * n$ fails for n large enough.

Big-O notation

Best bound

Note: Many upper bounds can be true for the same function — but we always report the tightest (best) one.

If $T(n) = 2n^2$, then

- $T(n) = O(n^4)$
- $T(n) = O(n^3)$
- $T(n) = O(n^2)$

are all technically correct, but the last option is the best answer.

Analysis of iterative algorithms

Example: iterative Fibonacci

```
ant = 1;           --> 1
act = 1;           --> 1
while (n > 2) {     --> n-2 + 1
    aux = ant + act; --> n-2
    ant = act;      --> n-2
    act = aux;      --> n-2
    n = n - 1;      --> n-2
}                  --> n-2+1
printf("%d", act); --> 1
```

$$T(n) = 6n - 7$$

Hence, the order of the algorithm is: $O(n)$

Error?, It should be $n-2$

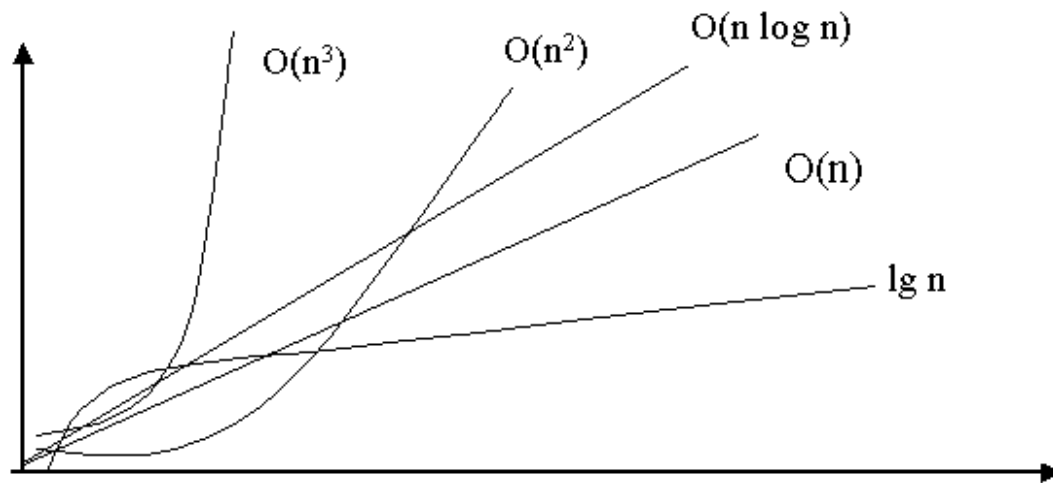
$$T(n) = 6n - 8$$

It doesn't affect our asymptotic analysis, the result is still

$O(n)$

Big-O notation

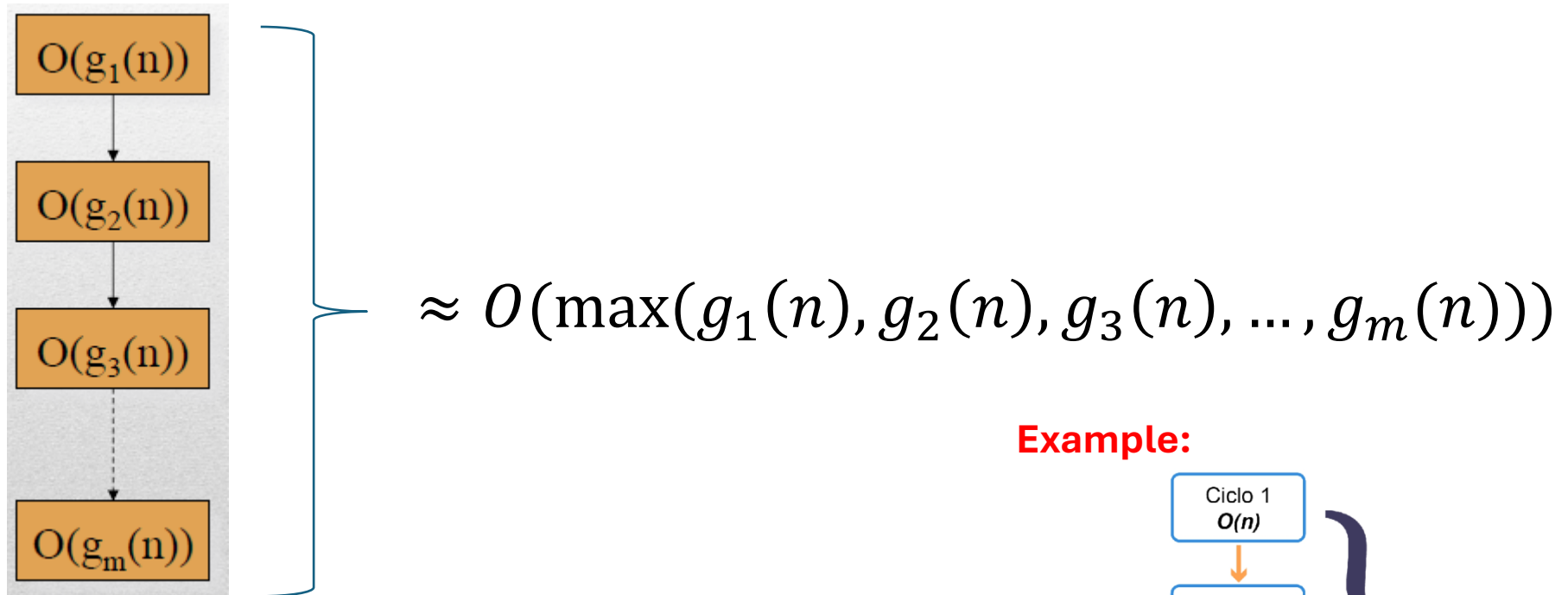
Typical complexities



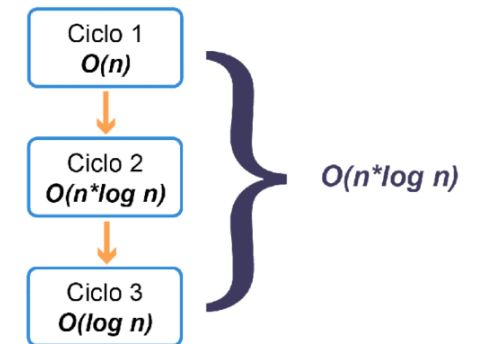
Big-O notation	Growth
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Log-linear
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(n^c)$	Polynomial ($c=\text{cte}$)
$O(m^n)$	Exponential ($m=\text{cte}$)
$O(n!)$	Factorial

Analysis of iterative algorithms

Consecutive Statements Rule

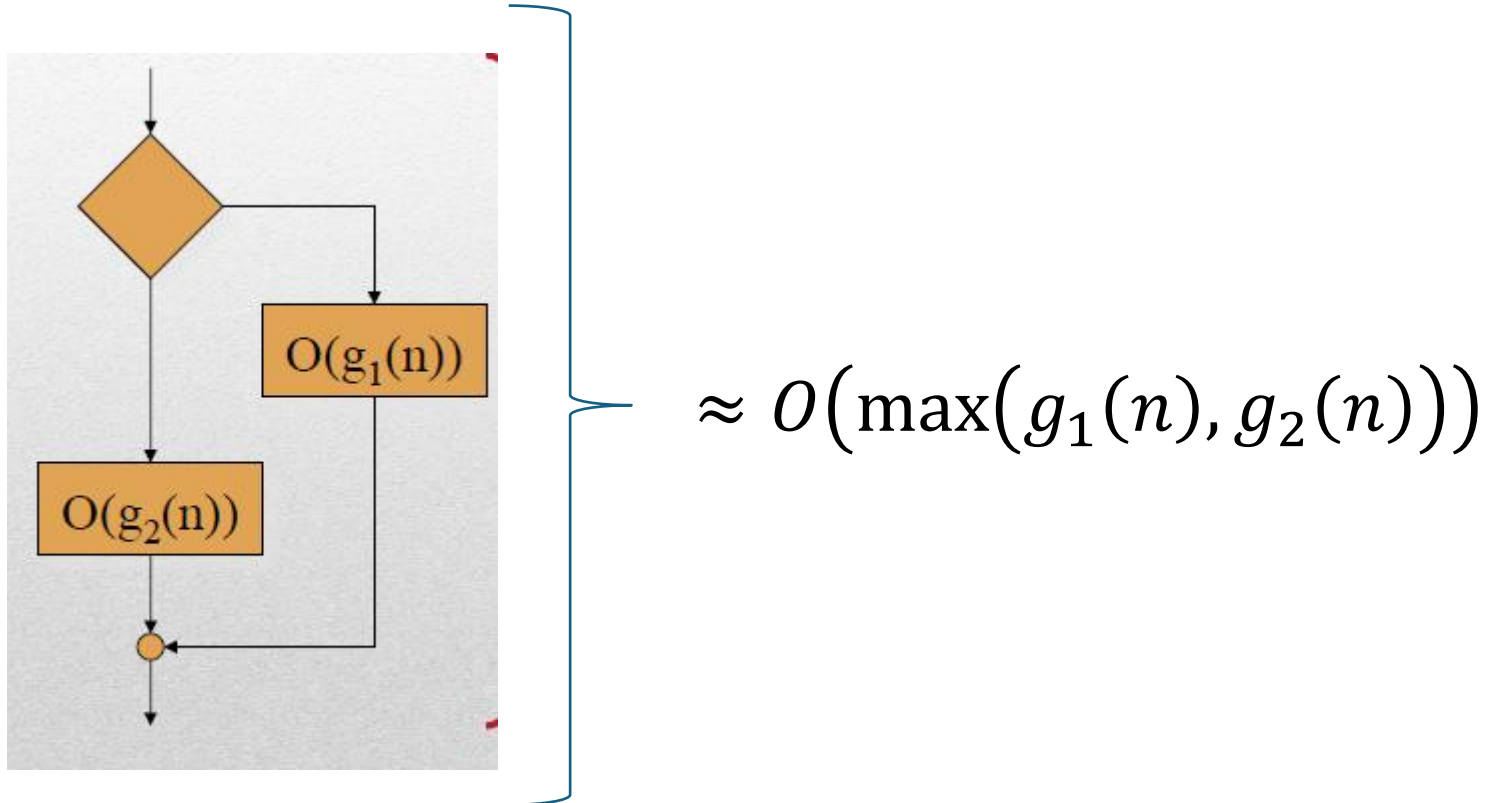


Example:

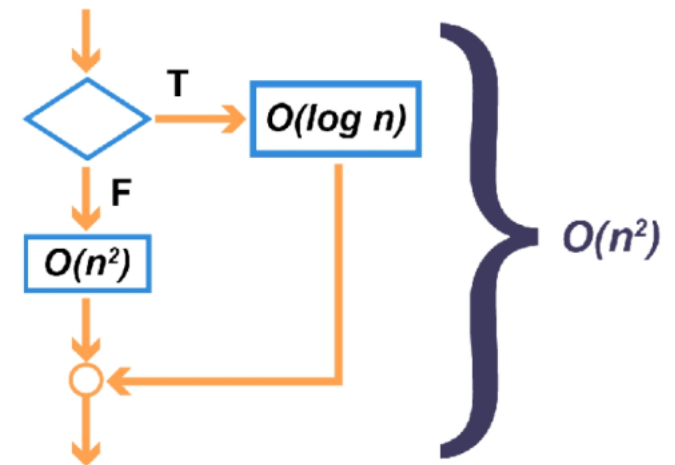


Analysis of iterative algorithms

if/else Rule

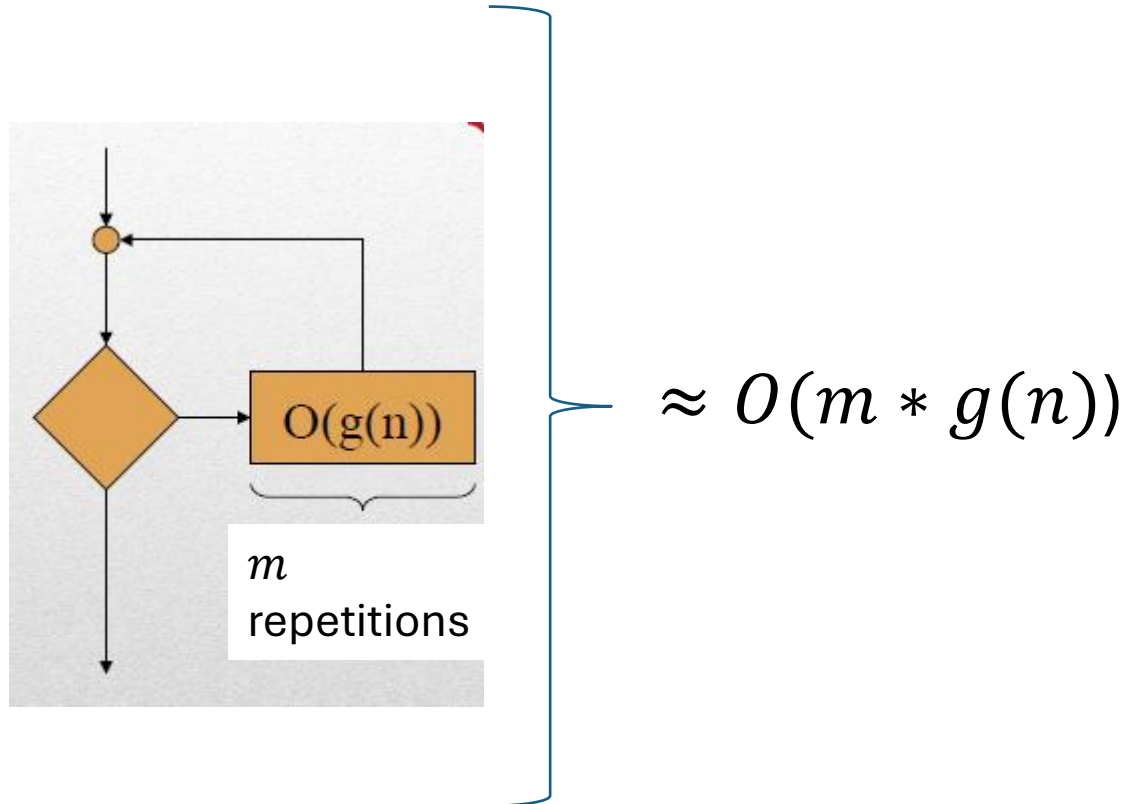


Example:

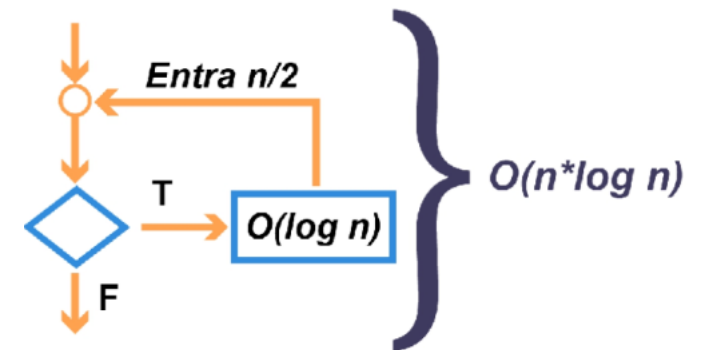


Analysis of iterative algorithms

Loops Rule



Example:



Analysis of iterative algorithms

Shortcuts for loops

Lineal → When the control variable is incremented or decremented by a constant.

```
1. for (int i=0; i<n; i++) {  
2.   instruccion;  
3. }
```

$O(n)$

Logarithmic → When the control variable is multiplied or divided by a constant.

```
1. for (int i=1; i<=n; i*=2) {  
2.   instruccion;  
3. }
```

$O(\log n)$

Analysis of iterative algorithms

Nested-loops Rule

- Analyse these inside out. The total running time of a statement inside a group of nested loops is the running time of the statement multiplied by the product of the sizes of all the loops.
- Example:

```
for( i = 0; i < n; ++i )  
    for( j = 0; j < n; ++j )  
        ++k;
```



$$O(n * n) = O(n^2)$$

Analysis of iterative algorithms

Special considerations

- In decisions and nested loops:
 - Analyse the code from inner to outer
- Tip for loops
 - What is the typical order of an internal instruction?
 - If the control variable increases or decreases with a constant value: linear order
 - If the control variable is multiplied or divided by a constant value: logarithmic order.

Analysis of iterative algorithms

Example: Exchange sort algorithm

```
for (i=1; i<n; i++)  
  for (int j=i+1; j<=n; j++)  
    if (a[j] < a[i])  
      intercambia(a[i], a[j]);
```

$\rightarrow O(1)$

$\left. \begin{array}{l} \rightarrow O(1) \\ \rightarrow O(1) \end{array} \right\} \rightarrow O(1)$

If-else rule -> maximum of the two branches

Analysis of iterative algorithms

Example: Exchange sort algorithm

```
for (i=1; i<n; i++)  
  for (int j=i+1; j<=n; j++)  
    if (a[j] < a[i])  
      intercambia(a[i], a[j]);
```

Worst case* repeats $n - 1$ times

$\left. \begin{array}{l} \text{if (a[j] < a[i])} \\ \text{intercambia(a[i], a[j]);} \end{array} \right\} \rightarrow O(1)$

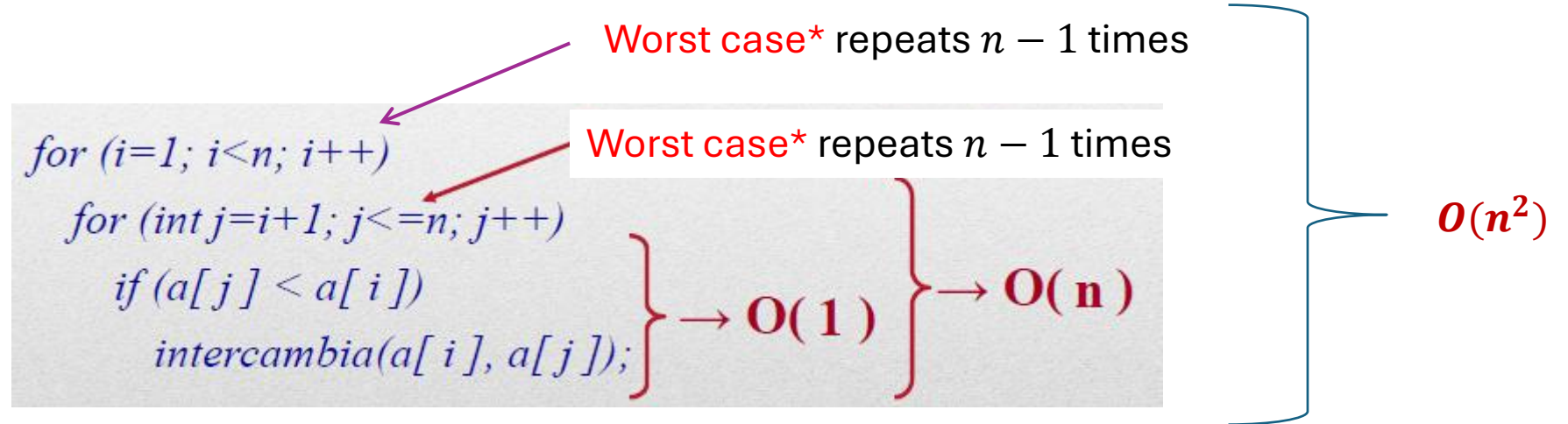
$\left. \begin{array}{l} \text{for (int j=i+1; j<=n; j++)} \\ \left\{ \begin{array}{l} \text{if (a[j] < a[i])} \\ \text{intercambia(a[i], a[j]);} \end{array} \right\} \end{array} \right\} \rightarrow O(n)$

Loop rule $\rightarrow$ (# repetitions)(order of the inner part)

* We are interested in the worst case

Analysis of iterative algorithms

Example: Exchange sort algorithm



Nested-loop rule

* We are interested in the worst case

What about recursive algorithms?

We'll cover that in the next class.

Analysis of iterative algorithms

Let's do some exercises!

References

M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed.
Boston, MA, USA: Pearson, 2014. Chapter 2.

V. M. de la Cueva Hernández, L. H. González Guerra, y E. G. Salinas
Gurrión, *Estructuras de datos y algoritmos fundamentales*.
Monterrey, México: Editorial Digital del Tecnológico de Monterrey,
2020.